Université Saad Dahlab BLIDA 1
Faculté des sciences
Département d'informatique

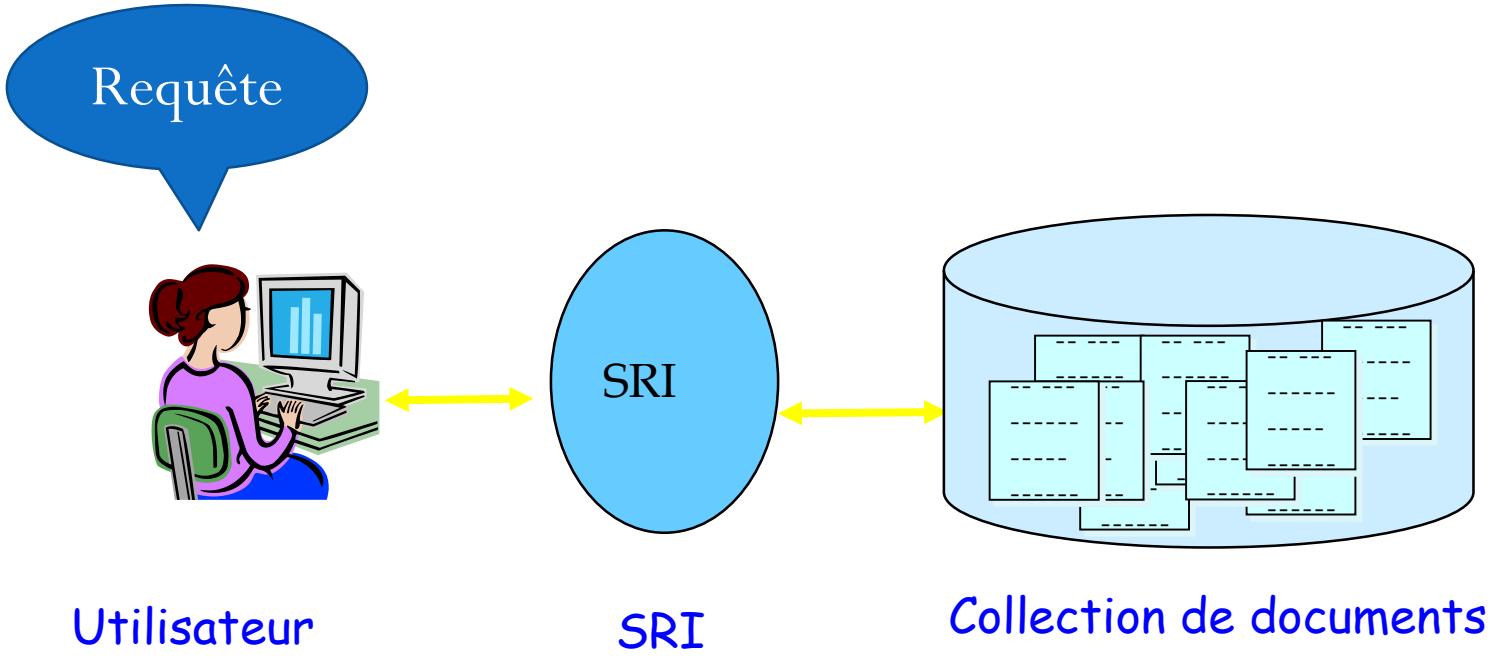
Chapitre 2: Indexation

Module: Recherche d'Information

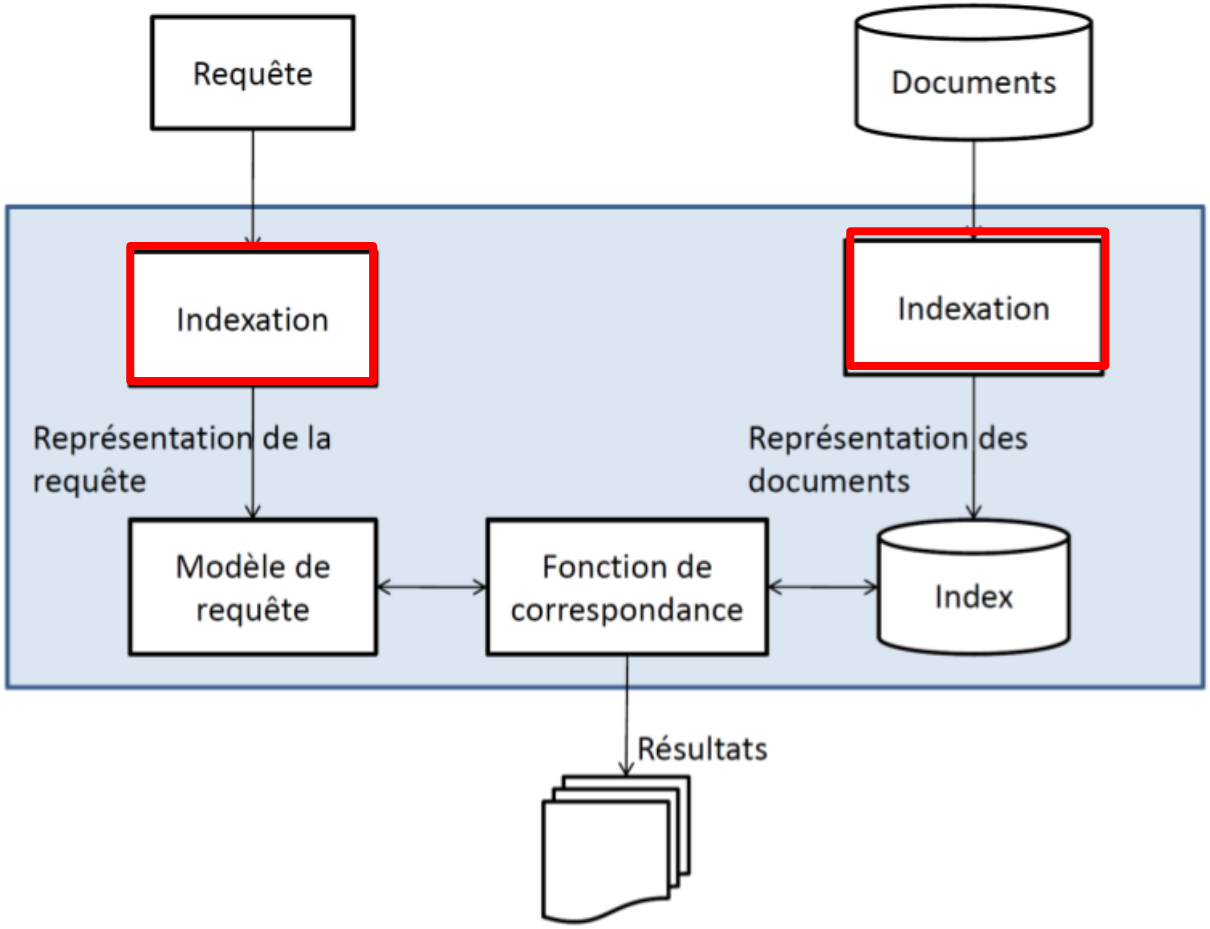
L3 ISIL

Par: Dr. Oukid L

Rappel



Rappel (Suite)



Architecture générale d'un système de Recherche d'Information

Indexation

- Processus de représentation des documents et de la requête
- Extraction des descripteurs (mots-clefs, termes)
- Représentation des descripteurs par une liste de termes significatifs pour l'unité textuelle correspondante

Objectif de l'indexation

- Trouver une structure permettant de représenter un document et une requête de façon à:
  - Garder que les informations pertinentes pour la recherche
  - Réduire la taille du document
  - Faciliter le processus de recherche
  - Réduire le temps de recherche

Indexation de documents

- **Peut être:**
  - Manuelle → Expert en indexation
  - Automatique → Programme informatique
  - Semi-automatique → Combinaison des deux

Indexation de documents (suite)

- **Manuelle :** nécessite l'intervention d'un expert de domaine afin de choisir les descripteurs qui représentent le mieux le contenu des documents
  - Méthode très couteuse en temps

- **Automatique :** s'effectue en utilisant un processus automatisé
  - Moins coûteuse que l'indexation manuelle.
  - Les descripteurs obtenus ne sont pas toujours pertinents.

- **Semi-automatique** : combine l'indexation manuelle et l'indexation automatique.
- Deux étapes :
  - Indexation automatique
  - Intervention d'un spécialiste de domaine

Processus d’indexation automatique

Etape 1: Extraction des mots

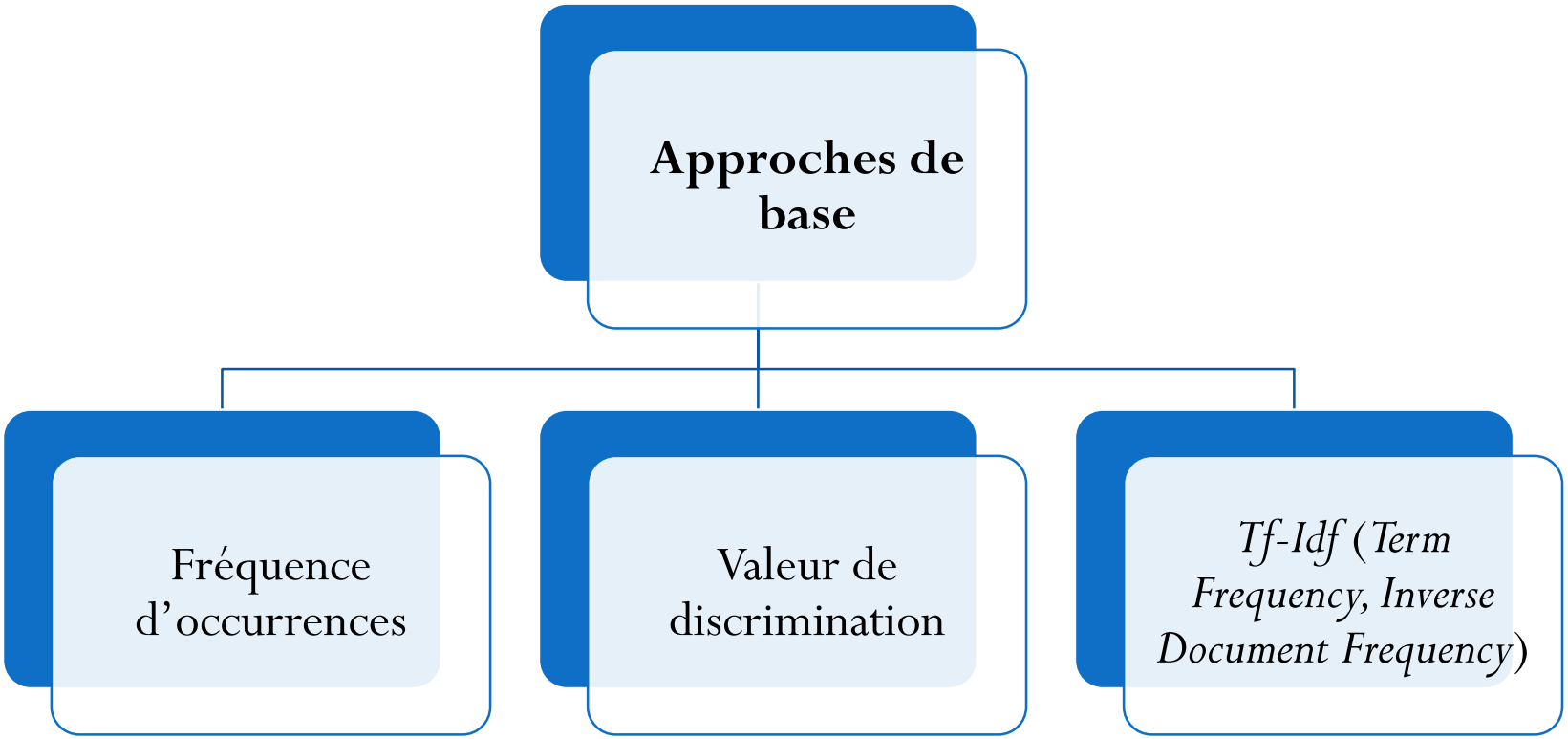
Extraire les termes « tokenization »

- **Terme** : suite de caractères séparés par (blanc ou signe de ponctuation, caractères spéciaux,…), Nombres

- Ce sont les index utilisés lors de la recherche

- **Dépend de la langue**
  - Exemple: langue française
    - L'ensemble → un terme ou deux termes ?
    - L ? L' ? Le ?

Indexation automatique



Approche basée sur la fréquence d'occurrences

- **Objectif :** retrouver les mots qui représentent le mieux le contenu d'un document

- **Principe :** Un mot est important si sa fréquence d'apparition dépasse **un seuil défini**

- **Etapes :**
  1. Calculer la fréquence d'apparition de chaque terme dans le document
  2. Définir un seuil minimal
  3. Garder uniquement les termes dont la fréquence est supérieure au seuil

Exemple 1

- Soit la collection de documents suivantes:

D1: « langage Java basé langage C++ Java langage puissant»

D2: « langage programmation C++ langage utilisé traduire algorithme programme langage Java C++ Python »

D3: « langage programmation Python très utilisé traitement texte programmation Python»

- Créer l'index de cette collection en utilisant la méthode de fréquence avec un seuil égale à 2.

Approche basée sur la valeur de discrimination

- **Objectif :** retrouver les termes qui distinguent un document par rapport aux autres documents de la collection

- **Principe :**
- Un terme est important s'il apparait seulement dans un petit nombre de documents
- Approche généralement utilisée: ***Tf-Idf***

Approche basée sur *Tf-Idf*

- ***TF « Term Frequency »:*** Fréquence d'un terme dans un document
  - Plus un terme est fréquent dans un document plus il est important dans la description de ce document

- ***IDF « Inverse Document Frequency »:*** Fréquence inverse d'un terme
  - Mesure l'importance d'un terme dans le corpus de documents

Calcul de Tf-Idf

- **TF**

$$Tf_{t,d} = \frac{n_{t,d}}{N_d}$$

  - Où $n_{t,d}$ est la fréquence d'apparition du terme t dans le document d et N_d est le nombre total des termes dans d

- **IDF**

$$Idf_t = \log \frac{D}{\{d_j : t_i \in d_j\}}$$

  - Où D est le nombre total de documents dans la collection et $\{d_j : c_i \in d_j\}$ représente le nombre de documents où le terme t apparait.

$$Tf - Idf = Tf_{t,d} \times Idf_t$$

Exemple 2

- Soit la collection de documents suivantes:

D1: « langage Java basé langage C++ Java langage puissant»

D2: « langage programmation C++ langage utilisé traduire algorithme programme »

D3: « langage de programmation Python est très utilisé pour le traitement de texte programmation Python»

- Créer l'index de cette collection en utilisant la méthode *Tf-Idf*

Amélioration du processus d’indexation automatique

Amélioration 1: Elimination des mots vides

- Les mots vides sont généralement les plus fréquents dans un document

- **Exemple:**
  - **Anglais :** the, or, a, you, I, us, …etc.
  - **Français :** le , la de , des, je, tu, …etc.

- Elimination des mots vides en utilisant une liste « ***StopList*** »

Exemple

- Soit le document suivant:

D1: « le langage java est basé sur le langage C++ »

- Mots vides dans D1:

D1: « le langage java est basé sur le langage C++ »

- Résultat après élimination des mots vides:

D1: « langage java basé langage C++ »

Amélioration 2 : Normalisation

- **Normalisation:**
  - Processus morphologique permettant de regrouper les variantes d'un mot

- **Techniques:**
  - Lemmatisation
  - Racinisation
  - Troncature

Normalisation (suite)

1- Lemmatisation

- Transformer les flexions en leur lemme
- Exemple: pris, prend, prisse → **prendre**

- **Outils logiciels:**
  - **Exemple:** TreeTagger

Normalisation (suite)

2- «Racinisation » (radicalisation) / (stemming)

- Transformer les flexions en leur radical ou stemme

- **Exemple :**
  - **Français:** économie, économiquement, économiste,

    ➡ **econom**
  - **Anglais** : retrieve, retrieving, retrieval, retrieved, retrieves

    ➡ **retriev**

PRÉFIXE	RADICAL	Suffixe
Pré	**traite**	**ment**

Normalisation (suite)

2- «Racinisation » (radicalisation) / (stemming)

- Utilisation de règles de transformations
  - règle de type : **condition action**
  - Exemple : **si** mot se termine par **s** **alors** supprimer la terminaison

  - Technique utilisée principalement pour l'anglais
    - L'algorithme le plus connu est : ***Porter***

Algorithme de Porter

- **Porter:** basé sur un ensemble de conditions actions
  - old suffix → new suffix
  - Les règles sont divisées en étapes et sont examinées en séquence
  - <u>e.g. Step 1a:</u>
    - sses → ss
    - ies → i *(ponies → poni)*
    - s → NULL *(cats → cat)*
  - <u>e.g. Step 1b:</u>
    - if m>0 eed → ee *(agreed → agree)*
    - if *v*ed → NULL *(plastered → plaster but bled → bled)*

  - http://www.tartarus.org/~martin/PorterStemmer/

Exemple de normalisation avec l'algorithme de Porter

- **Texte original:**

  marketing strategies carried out by U.S. companies for their agricultural chemicals, report predictions for market share of such chemicals, or report market statistics for agrochemicals, pesticide, herbicide, fungicide, insecticide, fertilizer, predicted sales, market share, stimulate demand, price cut, volume of sales

- **Texte après Porter + suppression des mots vides:**

  Market **4**, strateg **1**, carr **1**, compan **1**, US **1**, agricultur **1**, chemic **2**, report **2**, predict **2**, share **1**, statist **1**, agrochem **1**, pesticid **1**, herbicid **1**, fungicid **1**, insecticid **1**, fertil **1**, sale **2**, stimul **1**, demand **1**, price **1**, cut **1**, volum **1**

Normalisation (suite)

3- Troncature

- Tronquer les mots à X caractères
  - Tronquer plutôt les suffixes
  - Exemple troncature à 7 caractères
    - économiquement : écomoni

Quelle est la valeur optimale de X ? : 7 caractères pour le Français

Exemple de normalisation par troncature

- **Document**
  - un système de recherche d 'informations (document) (SRI, base de données documentaires) permet d 'analyser, d 'indexer et de retrouver les documents pertinents répondant à un besoin d 'un utilisateur.
- **Extraction des mots et élimination des mots vides**
  - système recherche informations document SRI base données documentaires analyser indexer retrouver document pertinents répondant besoin utilisateur
- **Normalisation par troncature à 7 caractères**
  - système recherc informa documen sri base donnee documen analyse indexer retrouv documen pertine reponda besoin utilisa
- **Pondération par la méthode de fréquences**
  - systeme 1,recherc 1, informa 1, documen 3, sri 1, base 1, donnee 1, analyse1, indexer 1, retrouv 1, pertine 1, reponda 2, besoin 3, utilisa *1*

Exemple 2

- Soit la collection de documents suivantes:

D1: « langage Java basé langage C++ Java langage puissant»

D2: « langage programmation C++ langage utilisé traduire algorithme programme »

D3: « langage de programmation Python est très utilisé pour le traitement de texte programmation Python»

- Créer l'index de cette collection après lemmatisation et pondération des termes en utilisant la méthode *Tf*.

Avantages \ inconvénients de la normalisation

- Contrairement à la lemmatisation, la stemmatisation produit des fois des "stems" qui n'ont pas de sens donc difficiles à interpréter
  - **Exemples:**
    - Porter: iteration/iter et general/gener
    - Troncature: Internet/Interne
- La stemmatisation est moins sensible aux fautes d'orthographes que la lemmatisation
  - La lemmatisation échoue à la moindre faute d'horthogaphe
- Stemmatisation: oublis de quelques normalisations intéressantes
  - **Exemple :**
    - European/Europe, matrices/matrix, machine/machinery ne sont pas normalisés

Références

-G. Salton and M. J. McGill. Introduction to modern information retrieval. McGraw-Hill, New York, 1983

-C. Manning, P. Raghavan, and H. Schütze. An Introduction to Informa-
-tion Retrieval. Cambridge university press, Cambridge, England, 2009.

-M.Boughanem. Cours Recherche d'Information
https://www.irit.fr/~Mohand.Boughanem/